

The μ OS++ Harness and Tests — a Synthesis

The μ OS++ harness and tests — a synthesis

- 0. In one paragraph
- 1. The folder
- 2. What lives where
- 3. The harness, in pieces
- 4. Which library is under test
- 5. The two kinds of tests
- 6. Step by step — from the folder to a passing test
- 7. Step by step — add a harness suite
- 8. Step by step — add a board test
- 9. Step by step — add a platform (a new board)
- 10. Rules you must not break
- 11. File map

The µOS++ harness and tests — a synthesis

Earlier layout — read as history. This document describes an earlier harness layout. The SMP kernel and test harness are now unified in `micro-os-plus-iii` (branch `smp`) alongside sibling repositories `micro-os-plus-iii-{cortexm,aarch32,aarch64,devices,posix-arch}`. For the current framework see `STEPS.md`; for every test, board and probe see `TESTS-CATALOG.md`. The code is the truth where they disagree.

The whole paradigm in one place: the repositories, the harness, how a test is chosen and built, and the steps from a folder of clones to a passing run. This is the map; the long-form walk-through is `WORK-SMP-AARCH32-AARCH64-HARNESS-GUIDE.md`, the per-file recipe is `HARNESS-BOARD-TEST-CHEATSHEET.md`, and the xPack plumbing is `TESTS-XPack-SYSTEM.md`.

0. In one paragraph

The **kernel** and the **test harness** live together in `micro-os-plus-iii-smp.git`; the harness is its `tests/` folder. A **port** (`-aarch32`, `-aarch64`, `-cortexm`) carries the ISA-specific code and its own board tests under `test/<board>/`. The harness has one **platform** folder per board (`tests/platforms/<platform>/`), each of which (a) says which library is under test, (b) defines the platform's compile options and support code, and (c) turns the portable **suites** (`tests/sources/`) and the port's **board tests** into executables and CTest cases. You drive it with xPack: `install → prepare → build → test`, one configuration per platform and build type. Everything runs under QEMU, on a probe, or natively, from the same sources.

1. The folder

The paradigm starts from a folder holding the repositories as clones, each named with its `.git` suffix:

workspace/	
micro-os-plus-iii-smp.git	the SMP kernel + the harness (tests/) + docs/
micro-os-plus-iii-aarch32.git	the AArch32 port + its board tests
micro-os-plus-iii-aarch64.git	the AArch64 port + its board tests
micro-os-plus-iii-cortexm.git	the Cortex-M port + its board tests
micro-os-plus-iii-devices.git	the shared devices package

Each clone's `origin` is a **bare** repository (here under `/home/dan/Downloads/GIT/*.git`). The bare is the meeting point: a working copy (the TMP7 folder) pushes to it, and these clones pull from it.

The **relative names are part of the contract**: the harness looks for the ports as `../../micro-os-plus-iii-aarch32.git` and `../../micro-os-plus-iii-aarch64.git` — siblings of the folder that holds the harness repo. Rename a clone and you must pass the new path (`-D UOS_AARCH32_DIR=...`); see §9.

2. What lives where

repository	carries	its tests
<code>micro-os-plus-iii-smp.git</code>	the kernel, and <code>tests/</code> — the harness	<code>tests/sources/</code> suites, <code>tests/platforms/</code>
<code>micro-os-plus-iii-aarch32.git</code>	the AArch32 port	<code>test/<board>/</code> (board tests)
<code>micro-os-plus-iii-aarch64.git</code>	the AArch64 port	<code>test/<board>/</code>
<code>micro-os-plus-iii-cortexm.git</code>	the Cortex-M port	<code>test/<board>/</code>
<code>micro-os-plus-iii-devices.git</code>	the device drivers	— (linked by the ports)

The rule that avoids most confusion:

- a **suite** (portable, runs on many boards) lives in the **harness**;
- a **board test** (needs this board's silicon) lives in the **port**.

3. The harness, in pieces

`micro-os-plus-iii-smp.git/tests/`:

```
tests/
  CMakeLists.txt      the project; sets C++20, enables testing
  cmake/
    common-options.cmake  micro-os-plus::common-options (flags, C++20)
    tests-main.cmake      the fixed sequence (see §4)
  platforms/<platform>/  one folder per board (see §3.2)
  sources/<suite>/        the portable suites (see §3.3)
  package.json          the xPack configurations and actions (see §3.4)
```

3.1 `tests-main.cmake` — the fixed sequence

Every configuration runs the same five steps, in this order:

1. `include cmake/common-options.cmake` — the shared compile/link options.
2. `include platforms/${PLATFORM_NAME}/cmake/definitions.cmake` — the platform's definitions (and, for a board, which `BOARD` the port is).

3. `include platforms/${PLATFORM_NAME}/cmake/dependencies-folders.cmake` — which suites and packages to `add_subdirectory()`.
4. **select and add the library under test** (the port, or the kernel).
5. `add_subdirectory(platforms/${PLATFORM_NAME})` — the platform's targets and CTest cases.

3.2 `platforms/<platform>/` — one folder per board

```
platforms/<platform>/
cmake/definitions.cmake      PLATFORM_NAME's defines; for a board, BOARD=
cmake/dependencies-folders.cmake the suites/packages to add_subdirectory()
cmake/platform-library.cmake  micro-os-plus::platform (+ -support)
CMakeLists.txt               the targets and the CTest cases
include/cmsis-plus/platform.h the platform's compile-time contract
src/platform-support.cpp      startup hooks + a strong main() (harness suites)
```

`platform-library.cmake` splits the platform into two libraries:

- `micro-os-plus::platform` — the base: flags, linker script, port glue.
- `micro-os-plus::platform-support` — the hooks plus a **strong** `main()` and the kernel C-library groups. A harness suite links **base + support**; a port board test links **base only**, because it brings its own `main()`.

3.3 `sources/<suite>/` — the portable suites

Each suite is a folder with a `CMakeLists.txt` that exports an INTERFACE library and nothing else:

```
add_library (test-mutex-stress-interface INTERFACE EXCLUDE_FROM_ALL)
target_include_directories (test-mutex-stress-interface INTERFACE "include")
target_sources (test-mutex-stress-interface INTERFACE src/main.cpp src/test.cpp)
add_library (test::mutex-stress ALIAS test-mutex-stress-interface)
```

A suite supplies **no** `main()` **and no startup hooks** — that is what `platform-support` is for. The suites are `blinky`, `cmsis-os-validator`, `fp-switch`, `instrumentation`, `mutex-stress`, `rtos-apis`.

3.4 `package.json` — the configurations and actions

Each `buildConfiguration` is one `(platform, buildType)` pair and carries the xPack commands:

property	command
<code>commandCMakeReconfigure</code>	<code>cmake -S . -B <build> -G Ninja -D CMAKE_BUILD_TYPE=... -D PLATFORM_NAME=...</code>
<code>commandCMakePrepareWithToolchain</code>	the reconfigure plus <code>-D CMAKE_TOOLCHAIN_FILE=...</code>
<code>commandCMakeBuild</code>	<code>cmake --build <build></code>
<code>commandCMakePerformTests</code>	<code>cd <build> && ctest -V</code>

and the actions:

```
"actions": {
  "prepare": "{{ properties.commandCMakePrepareWithToolchain }}",
  "build": [ "{{ properties.commandCMakeReconfigure }}",
             "{{ properties.commandCMakeBuild }}" ],
  "test": "{{ properties.commandCMakePerformTests }}",
  "test-mutex-stress": "{{ properties.commandCMakePrepareWithToolchain }} &&
{{ properties.commandCMakeBuild }} && cd {{ properties.buildFolderRelativePath }} && ctest -V -R
mutex-stress"
}
```

The `test-*` actions configure **with** the toolchain (`PrepareWithToolchain` , not the bare `Reconfigure`) so a rebuild uses the pinned compiler; a *fresh* configuration still needs `xpm run install` once.

4. Which library is under test

`tests-main.cmake` picks it from `PLATFORM_NAME` :

<code>PLATFORM_NAME</code> starts with	library under test
<code>aarch32</code>	<code>micro-os-plus-iii-aarch32.git</code> (<code>add_subdirectory</code>)
<code>aarch64</code>	<code>micro-os-plus-iii-aarch64.git</code>
<code>native</code>	<code>micro-os-plus-iii-posix-arch.git</code>
<code>cortexm</code> , <code>qemu-cortex</code>	<code>micro-os-plus-iii-cortexm.git</code>
<code>pico2</code> , <code>2xcortex</code>	<code>micro-os-plus-iii-cortexm.git</code>
anything else	the kernel itself (<code>add_subdirectory("../")</code>)

So the `qemu-cortex-*` , `cortexm-*` , `pico2-1cpu` and `2xcortex-m33` platforms test the local Cortex-M port, the `aarch32-*` / `aarch64-*` platforms the ARM ports, and `native` the POSIX-arch port; only `nucleo-*` and `raspberrypi-pico` fall through to the kernel itself.

Add the kernel only once. The port already adds the kernel (and the devices package), so a platform must not `add_subdirectory()` them too — the duplicate `micro-os-plus::iii` alias fails the configure.

5. The two kinds of tests

	harness suite	board test
lives in	<code>tests/sources/<name>/</code>	<code><port>/test/<board>/<name>/</code>
supplies	no <code>main()</code> , no hooks	its own <code>main()</code> + startup hooks
links	<code>platform</code> + <code>platform-support</code>	<code>platform</code> only
portable to	any platform	one board
registered by	the platform's <code>CMakeLists.txt</code>	<code>add_hw_test()</code> in the platform

A board test can be hardware-only: a platform whose `board.cmake` sets no `UOS_BOARD_LINKER_QEMU` gets one image per test (`hwd`) and no emulated suite.

6. Step by step — from the folder to a passing test

```
# cwd: workspace/micro-os-plus-iii-smp.git/tests

# 1. the top-level tools (cmake, ninja, build-helper, validator, chan-fatfs)
npm install
xpm install

# 2. the pinned cross tools for THIS configuration
xpm run install --config aarch32-rpi-zero-2w-cmake-gcc-debug

# 3. configure — this is where -D CMAKE_TOOLCHAIN_FILE is applied
xpm run prepare --config aarch32-rpi-zero-2w-cmake-gcc-debug

# 4. build
xpm run build --config aarch32-rpi-zero-2w-cmake-gcc-debug

# 5. run (one test, or the whole configuration)
xpm run test-mutex-stress --config aarch32-rpi-zero-2w-cmake-gcc-debug
xpm run test                --config aarch32-rpi-zero-2w-cmake-gcc-debug
```

Why the order: the pinned toolchain is installed **inside the build folder** (`build/<config>/xpacks/.bin`), so `install` must precede `prepare` / `build` , and `build/<config>/` must not be deleted. Skipping `install` is the usual cause of `found /usr/bin/cc` from a platform's version guard.

The `test-*` actions do `prepare` + `build` themselves, so a plain `xpm run test-<name>` rebuilds first — but a configuration that has never been `install`ed still needs that once.

7. Step by step — add a harness suite

1. **The suite** — `tests/sources/<name>/` with `include/`, `src/`, and a `CMakeLists.txt` exporting `test::<name>` (copy `sources/mutex-stress/`). No `main()`.
 2. **The platform** — add the folder to `platforms/<platform>/cmake/dependencies-folders.cmake`:
`cmake set (xpack_dependencies_folders "${CMAKE_SOURCE_DIR}/sources/<name>" ...)`
 3. **The target + test** — in `platforms/<platform>/CMakeLists.txt`:
`cmake add_executable (<name>-test) target_link_libraries (<name>-test PRIVATE micro-os-plus::common-options test::<name> micro-os-plus::iii micro-os-plus::platform micro-os-plus::platform-support) # output name, runtime dir, objcopy, then: add_test (NAME "${PLATFORM_NAME}-<name>-test" COMMAND bash "${_port_hw}" <name>-test 600)`
 4. **The action** — `package.json`, one `test-<name>` per configuration:
`json "test-<name>":
 "{
 properties.commandCMakePrepareWithToolchain
 }" &&
 "{ properties.commandCMakeBuild }" && cd "{ properties.buildFolderRelativePath }" &&
 ctest -V -R <name>`
-

8. Step by step — add a board test

1. **The test** — in the **port**: `<port>/test/<board>/<name>/main.cpp`, with its own `main()` and startup hooks (copy a sibling like `smp_test5/`).
2. **The port's registration** — `<port>/test/<board>/tests.cmake`: add it to `BOARD_TEST_NEED_DEVICES` / `BOARD_TEST_SELF_CONTAINED`, and, if it needs sources beyond its own folder, to `board_test_sources()` / `board_test_includes()`.
3. **The harness** — one `add_hw_test()` in `platforms/<platform>/CMakeLists.txt`, reading the sources from the port:
`cmake add_hw_test (<name> SOURCES "${_port_test_dir}/<name>/main.cpp" INCLUDES "${_port_test_dir}/include" "${_port_test_dir}/<name>" [LIBRARIES micro-os-plus::devices-rk3506] [SECONDS 900])`
4. **The action** — a `test-<name>` in `package.json`, as in §7.

The harness reads `<port>/test/<board>/` from the **port clone** — write and commit the test in the port's working copy, push, and pull it into the clone the harness builds from.

9. Step by step — add a platform (a new board)

A platform is a folder, plus its configuration:

1. `tests/platforms/<name>/cmake/definitions.cmake` — set `xpack_platform_compile_definition`, and (for a port board) the `BOARD` cache variable the port reads.
2. `tests/platforms/<name>/cmake/dependencies-folders.cmake` — the suites this board runs.
3. `tests/platforms/<name>/cmake/platform-library.cmake` — the base and the support, and the C-library groups.
4. `tests/platforms/<name>/include/cmsis-plus/platform.h` — the compile-time contract.
5. `tests/platforms/<name>/src/platform-support.cpp` — the startup hooks and a strong `main()` (harness suites only).
6. `tests/platforms/<name>/CMakeLists.txt` — the targets and the CTest cases.
7. `tests/package.json` — a `buildConfiguration` per build type, with the platform name and the toolchain file.
8. If the platform tests a **port**, add the matching branch to `tests-main.cmake` (like `^aarch32`), or keep the kernel fallback.

If the clone names differ from `../../micro-os-plus-iii-aarch32.git`, pass the paths instead:

```
xpm run prepare --config <config> \  
-D UOS_AARCH32_DIR=/abs/path/to/micro-os-plus-iii-aarch32
```

10. Rules you must not break

- **Add the kernel once.** Only the port (or the fallback) may `add_subdirectory()` the kernel; a second add fails on the duplicate `micro-os-plus::iii` alias.
 - **A harness suite supplies no `main()`.** It links `platform-support`; a board test supplies its own `main()` and links only `platform`.
 - **install before prepare / build, and never delete `build/<config>/`** — it holds the pinned toolchain.
 - **Configure with the toolchain.** Use `commandCMakePrepareWithToolchain` in any action that configures, or a fresh tree picks the host compiler.
 - **Board tests are one per power cycle** on hardware; give them `LABELS hwd` and keep the generic `test` action to `ctest -LE hwd`.
 - **Tests come from the port clone**, not from `tests/`: `_port_test_dir` is `${UOS_AARCH32_DIR}/test/<board>`, and `add_hw_test()` enumerates them.
-

11. File map

```
workspace/
micro-os-plus-iii-smp.git/
  tests/
    CMakeLists.txt           the test project (C++20)
    cmake/common-options.cmake micro-os-plus::common-options
    cmake/tests-main.cmake   the fixed sequence (§3.1, §4)
    platforms/<platform>/    one folder per board (§3.2)
      cmake/{definitions,dependencies-folders,platform-library}.cmake
      include/cmsis-plus/platform.h
      src/platform-support.cpp
      CMakeLists.txt
    sources/<suite>/         the portable suites (§3.3)
      CMakeLists.txt        -> test::<suite>
      include/ src/
      package.json          configurations + actions (§3.4)
    docs/tests/             this document and its siblings
micro-os-plus-iii-aarch32.git/
  test/<board>/             board tests (§8)
    tests.cmake
    test/boards/<board>/    board.cmake, linker, OpenOCD, hw.sh
micro-os-plus-iii-aarch64.git/ the same shape
micro-os-plus-iii-cortexm.git/ the same shape
micro-os-plus-iii-devices.git/ the drivers the ports link
```

Companion to [TESTS-XPACK-SYSTEM.md](#), [WORK-SMP-AARCH32-AARCH64-HARNESS-GUIDE.md](#) and [HARNESS-BOARD-TEST-CHEATSHEET.md](#).